

What Won, and How We Win

All 16 ETHOnline 2026 winners and finalists close to our idea, what they did, and our build plan

The short version

Sixteen projects close to our idea won something at ETHOnline 2026: 4 finalists and 12 sponsor-prize winners.

None of them won with a spending cap alone. They won by enforcing limits where money actually moves, keeping keys away from the AI, proving safety with real attack tests, and telling a plain story with real numbers.

Our plan: a solid baseline of eight features (B1–B8) that every winner had some version of, built on Solana so the token program itself enforces the limit, plus a few standout features (X1–X8) that nobody at ETHOnline could show, led by “**turn off our server and Solana still says no.**”

How to read this: Part 1 is a one-page overview. Part 2 covers each project (what it set out to do, the problem, how it works, what made it stand out, and what we take). Part 3 lists the patterns shared by the winners. Part 4 is our plan: baseline, standout features, demo, timeline and risks.

Prepared 2 Oct 2026 from the full ETHGlobal submissions of each project. Figures quoted are the projects' own claims. WHU deadline: 5 Oct 2026, 00:00 CEST.

Part 1 • The 16 at a glance

F = finalist, D = agents paying for data or tools, A = controlling what an agent can do or spend.

Project	Group	Core idea	Result
OpenBook	F	Agent data market; stale data refunded by the contract	Finalist; Arc Best Agentic Economy App; The Graph 3rd
Cordon	F	One shared budget for a team of agents	Finalist
OnchainRouter	F	Paid on-chain tools; discover free, pay per call	Finalist
Novi Corpus	F	Legal company + guardian + capped treasury per agent	Finalist; World AgentKit Continuity
Turnstile	D	Agents buy analysis per call; method kept secret	Hedera AI & Agentic Payments; The Graph 2nd
AutoVoyage	D	Travel agent paying per query; approval bound to itinerary	Hedera AI & Agentic Payments
Carpool	D	Buy research already done instead of redoing it	Hedera AI & Agentic Payments
TrueTick	D	Tokenized stock prices checked across venues, sold per call	The Graph 3rd
Batas	A	Agent runs a liquidity position under limits it can't widen	ENS Best Use of ENSv2, 2nd
Capsule	A	Launch an agent in minutes; owner holds limits and off switch	ENS Best Use of ENSv2, runner-up
GhostKey	A	Scoped, expiring authority instead of secrets	Ledger AI Agents, 2nd
Harness	A	Machine + name + on-chain daily limit per agent	Ledger AI Agents, 1st
LedgerMind	A	Payment intent contract with caps instead of a key	Bazantic, Help an Agent Use Your Project
Mandate-app	A	Agent trades for a treasury without key or custody	Bazantic, Best Recipe, 2nd
NotYet	A	Budget released by period; future budget can't be stolen	Bazantic, Best Recipe, 1st
payOrRefuse	A	Refuse before signing unless the seller is proven	Bazantic, Help an Agent Use Your Project

What this tells us

12 of the 16 are about limiting what an agent can do or spend. It is the hottest theme, so a cap by itself will not stand out.

The data and tool winners (OpenBook, OnchainRouter, Carpool, Turnstile, TrueTick, AutoVoyage) all pair paying per call with one sharp promise: fresh or refund, no answer no charge, don't redo work.

Not one of them runs on Solana. They used Arc, Hedera, Sepolia and Sei.

Part 2 · Project by project

FINALISTS

1. OpenBook

Finalist; Arc Best Agentic Economy App; The Graph 3rd

Objective: A data marketplace for AI agents where stale data is refunded automatically by the contract.

Problem: Agents buy data at machine speed, some of it arrives stale, and there is no refund or anyone to call. The pitch backs this with numbers: bad data costs organisations \$12.9M a year (Gartner), 54% of organisations deploy agents (KPMG), 40%+ of agent projects will be cancelled for weak risk controls.

HOW IT WORKS

1. Seller lists a dataset under an ENS name with a price per query and a freshness promise.
2. Buyer asks in plain English; payment locks into an escrow job on Arc with the promise written in.
3. Data comes from The Graph stamped with its block number; the delivery is hashed and signed.
4. A hook on the escrow checks the block against the freshness floor: fresh pays the seller 98% (2% fee), stale reverts and refunds.

WHAT MADE IT STAND OUT

- Freshness is enforced at settlement, by the contract.
- Every settlement, refund and fee is indexed into public books.
- Price and payee are re-read from ENS before every job; no defaults in the code.
- 409 unit tests, 20 contract tests, a 20-check browser audit in CI; judges can buy with nothing installed.

What we take

A guarantee enforced at payment time and public books. On Solana: a receipt account per purchase, and a report anyone can rebuild from the chain.

2. Cordon

Finalist

Objective: One shared, enforceable budget for a whole team of AI agents and the helpers they spawn.

Problem: Separate allowances for each agent can add up to more than the owner intended; fixed allowances also strand money. Cites a June 2026 paper with 63 real budget-overflow incidents.

HOW IT WORKS

1. Owner sets a mandate for a task; helpers only ever get a narrower mandate.
2. Agent's only tool is `cordon_fetch(url)`. The daemon reads price and payee from the seller's 402, not from the agent.
3. Vault performs ancestor debit: a helper's \$1 is charged at every level up to the root.
4. Refusals are published to a reputation registry; owner can approve specific releases.

WHAT MADE IT STAND OUT

- No pay or transfer tool exists, and a test asserts it stays unregistered.
- No admin key, proxy or upgrade path.
- 1,200 adversarial strategies over 45,360 draws, none beat the bound; a hostile LLM with \$0.02 reached \$0.007.

- A planned three-way evaluation (no control, hard stop, Cordon) on cost and completion.

What we take

This is our mechanism. Copy the safety design (no pay tool, price from the 402) and the attack testing, and run it on Solana where the token program enforces the cap.

3. OnchainRouter

Finalist

Objective: A catalogue of paid on-chain tools that agents discover for free and pay per call.

Problem: Developers wire up many separate APIs; agents need to pay tiny amounts without accounts; agents need decisions, not tables.

HOW IT WORKS

1. Agent connects once over MCP or HTTP and sees every tool and price for free.
2. Calling a tool returns 402; the agent pays per call (HBAR on Hedera or USDC on Arc) straight to the tool author.
3. The router never settles a failed call (non-2xx or tool error): no answer, no charge.
4. One registry generates the catalogue, docs, tool pages and /llms.txt.

WHAT MADE IT STAND OUT

- No wallet held and no commission taken by the router.
- Outliers discarded against the median; stale markets dropped; coverage probed, not claimed.
- Claude Code gets a wallet as a second MCP server with one tool: sign_x402_payment.
- One npm call turns any function into a paid endpoint.

What we take

No answer, no charge and free discovery before paying. Ship llms.txt + MCP so judges and agents can try us in one step.

4. Novi Corpus

Finalist; World AgentKit Continuity

Objective: A Wyoming LLC for each AI agent, with a verified human guardian and a treasury the agent cannot overspend.

Problem: Agents that buy and contract need a legally accountable body; counterparties need to check it; owners need limits.

HOW IT WORKS

1. Guardian verifies with World ID before a company can be formed.
2. Company gets an on-chain identity and a treasury the guardian can pause.
3. Agent pays from an account that both agent and guardian control (1-of-2), so the guardian can rotate the agent out.
4. Other agents pay \$0.001 for a signed legal-standing check.

WHAT MADE IT STAND OUT

- A payment counts as settled only after reading it back from the ledger, never on the facilitator's word.

- Seller policy fails closed (503) if any check cannot be read.
- Signed answers can be verified offline.

What we take

A real-world business framing, verify settlement from the chain, and fail closed.

AGENTS PAYING FOR DATA AND TOOLS

5. Turnstile

Hedera AI & Agentic Payments; The Graph 2nd

Objective: A marketplace where agents buy on-chain analysis one call at a time, and the seller keeps its method secret.

Problem: Selling an answer usually means giving away how you got it; agents need a price they can read and trust.

HOW IT WORKS

1. Seller's offer lives in an ENS name: price, accepted chains, endpoint.
2. Buyer agent reads the price from the chain, calls, gets 402, pays on Hedera or Arc, receives the answer.
3. Agent spends from a wallet whose cap needs two signatures to raise.
4. Premium scorer runs in a Chainlink enclave.

WHAT MADE IT STAND OUT

- A four-method PaymentRail interface; a build test fails if payment code mentions any chain.
- Hot key can change price, not payout address: the chain reverts it, proven by a fork test.
- Agent signs offchain authorisations only: eleven payments, zero gas, nonce still 0.

What we take

A chain-agnostic payment interface (our contract already says chain sits behind one). **The agent cannot raise its own cap.**

6. AutoVoyage

Hedera AI & Agentic Payments

Objective: A travel agent that plans and books trips, paying for flight data per query from the user's own account.

Problem: Agents need data access without API keys, and users need to stop a bait-and-switch between quote and booking.

HOW IT WORKS

1. User signs once to grant an HBAR allowance with a budget (native Hedera allowance).
2. Agent pays the data supplier per result from the user's account; ceilings are checked before settling.
3. Before booking, the agent re-hashes the exact itinerary and needs a human confirmation bound to that hash.
4. Every payment, approval, booking and refusal is a public log entry.

WHAT MADE IT STAND OUT

- Non-custodial: money stays in the user's account; allowance revocable any time.
- Quote cache keyed by the query hash so the 402 price equals the charged price.
- Flight data openly labelled as fixtures; payments are real.

What we take

Approval bound to a hash of exactly what will be bought. On Solana: the Blink carries the purchase hash. **Native delegation**, which on Solana is SPL approve. **Label what is mocked.**

7. Carpool

Hedera AI & Agentic Payments

Objective: A registry where agents buy research someone already did instead of redoing it.

Problem: Agents burn time and tokens re-answering questions others have answered.

HOW IT WORKS

1. Before working, the agent searches Carpool by meaning; if an answer exists it pays a small USDC fee and skips the work.
2. Research is addressed by its content hash; price halves on an author-chosen schedule and stops after three halvings.
3. Buyers get a two-minute refund window; authors are paid in batches with a public Merkle root.
4. MCP with four tools; publishing asks the user and strips secrets first.

WHAT MADE IT STAND OUT

- Measured saving: redoing a prize analysis cost \$3.97 in tokens, buying it cost \$0.44.
- A simulation found a double-payout bug; a lease row fixed it; confirmed on testnet.
- A separate script re-checks 60 on-chain claims without importing project code.

What we take

Don't pay twice and a measured saving number. A short refund window is a cheap form of recourse.

8. TrueTick

The Graph 3rd

Objective: One table of tokenized-stock prices across issuers and chains, flagged against the real price, sold to agents per call.

Problem: The same stock exists on several chains and pools that disagree; one NVDA pool priced at \$1,153 vs a real \$218.

HOW IT WORKS

1. One normalised schema with one adapter per venue (including Solana Raydium).
2. Bad prices are labelled (phantom liquidity, market closed, thin volume), never dropped or set to zero.
3. Price service behind x402; the MCP server decides to spend and the key never reaches the model.

WHAT MADE IT STAND OUT

- Every price is a number or null, never 0.
- Checks the facilitator's identity before paying.
- Settlement happens after the handler and is cancelled on errors: a failed request costs nothing.
- Refuses to pay while a data source is cold, so it never pays for an answer the client will time out on.

What we take

Never pay for an answer you can't receive. Label bad data instead of hiding it. Check who you are paying before paying.

CONTROLLING WHAT AGENTS CAN DO AND SPEND

9. Batas

ENS Best Use of ENSv2, 2nd

Objective: Let an agent manage a 1inch Aqua liquidity position under limits it cannot widen, with a name you can revoke.

Problem: Most agent tooling enforces limits in the app layer, “which means nowhere, because anyone can call the contract directly”.

HOW IT WORKS

1. The mandate (size cap, floor price, expiry) is the strategy itself: its hash is the position's identity.
2. Limits are checked inside the contracts that move the money, before tokens leave.
3. The agent's authority is an expiring, revocable name; revoking it makes every trade revert for every caller.
4. Three of four questions about a position are free; a counterparty pays only when real doubt remains.

WHAT MADE IT STAND OUT

- Measured attack: same attacker, same reserves. With mandate: 2 trades, 83% of the position left. Without: 64 trades, gutted.
- 2,000-run fuzz found a one-second disagreement at the expiry boundary; fixed.
- A README test asks the chain about every address and transaction the README cites.

What we take

Enforce where the money moves and show the attack with and without protection. This is the single strongest demo idea in the set.

10. Capsule

ENS Best Use of ENSv2, runner-up

Objective: Launch an AI agent in 2–3 minutes, with its identity, spending cap and off switch held under your own name.

Problem: Platforms control agent identity and shutdown, and running your own agent needs DevOps skills.

HOW IT WORKS

1. A five-step no-code flow mints the agent a name, sets model and prompt, and starts it.
2. The agent writes a heartbeat; the owner revokes by removing that one permission, directly, without our backend.
3. Spending limits are records the owner can write and the agent cannot.

WHAT MADE IT STAND OUT

- **Two-process split:** the supervisor holds the key; the model's process is built with an empty environment and can only ask a local broker, which checks limits first.
- Fails closed: anything unreadable means zero, and zero means no.
- Reserves gas so an agent can never spend itself into looking revoked.

What we take

The model never touches the key (two processes) and a kill switch that works even if our server is gone.

11. GhostKey

Ledger AI Agents, 2nd

Objective: Give agents scoped, expiring authority instead of secrets.

Problem: Agents get long-lived API keys and credentials that prompt injection can leak.

HOW IT WORKS

1. Each agent gets a short-lived identity bound to a provider, resource, actions and expiry.
2. Real credentials stay encrypted and are used internally; the agent never sees them.
3. Destructive actions are refused before any provider call; escalation needs a physical Ledger confirmation.

WHAT MADE IT STAND OUT

- Separates authority from secrets: holding a capability does not reveal the credential.
- Elevated authority expires automatically.

What we take

Authority, not keys and **expiring elevated permission** after a human approves.

12. Harness

Ledger AI Agents, 1st

Objective: Every agent gets its own machine, name and an on-chain spending limit; one tap on a Ledger sets it up.

Problem: Giving an agent money means giving it a key; then only its judgement stops it.

HOW IT WORKS

1. One Ledger tap creates a container, mints a name and writes a daily limit on chain.
2. Inside the limit the agent acts freely; anything beyond goes back to the Ledger.
3. The registry measures the balance before and after a batch and reverts if it exceeds the allowance.
4. Revoking is one transaction: name, shell access and money all stop together.

WHAT MADE IT STAND OUT

- Credentials are decrypted only if the chain still says the agent is live.
- Four setup transactions batched into one confirmation.

What we take

Measure the actual balance change rather than trusting what the agent says it spent. **One revoke stops everything at once.**

13. LedgerMind

Bazantic, Help an Agent Use Your Project

Objective: Replace an agent's wallet key with a payment intent: a contract with caps the agent can only execute within.

Problem: A wallet key means unlimited authority, revoking means rotating keys, and auditing means trusting the agent's own logs.

HOW IT WORKS

1. Each intent enforces a total cap, a per-transaction cap, a time window and an optional merchant list.

2. Every payment emits a receipt hash committing to the context that justified it.
3. An assistant reads live indexed data and recommends actions: revoke this, withdraw that, restrict merchants.

WHAT MADE IT STAND OUT

- Arithmetic done in code before the model sees anything, so a cheaper model works.
- Measured A/B of their agent guide; honest about where it made no difference.
- No silent fallback: if the index is down, the API says so (503).

What we take

Receipt that commits to why the payment was made, and an assistant that recommends revokes from spending data.

14. Mandate-app

Bazantic, Best Recipe, 2nd

Objective: An agent can act for a treasury without holding its key or its funds.

Problem: Treasury automation usually hands an agent custody or a key.

HOW IT WORKS

1. Owner's instructions become an immutable on-chain strategy: who, which tokens and venue, minimum rate, caps, expiry.
2. On each call the contract re-checks everything, pulls only the approved input, swaps, measures the real output and returns it to the owner.
3. Any failed check reverts the whole transaction.

WHAT MADE IT STAND OUT

- Measures real balance changes instead of trusting return values.
- Route check returns PASS, FAIL or UNKNOWN; the route provider alone can never produce PASS.
- Clearly labelled test venue on testnet, plus a mainnet fork test for realism.

What we take

Three-state decisions (PASS / FAIL / UNKNOWN, and UNKNOWN refuses) and honest labelling of test setups.

15. NotYet

Bazantic, Best Recipe, 1st

Objective: Give an agent a budget on a schedule, where future budget does not exist yet even if the agent is hacked.

Problem: Hand over the whole budget and one hack drains it. Real 2026 drains: Grok/Bankr ~\$175K, Lobstar Wilde ~\$250K, malicious LLM routers ~\$500K.

HOW IT WORKS

1. Each period's spend key is time-lock encrypted to a future moment; before then nobody can use it.
2. A vault enforces the period window, budget and per-transaction cap.
3. Over-cap spends need a hardware co-signature.

WHAT MADE IT STAND OUT

- Worst case if hacked today: one period's budget, never the treasury.
- Opens with named real incidents and dollar amounts.

What we take

Blast radius as a headline: “hacked today = at most today's budget”. On Solana, period budgets through Squads spending limits or a daily-reset delegate.

16. payOrRefuse

Bazantic, Help an Agent Use Your Project

Objective: An x402 payment gate that refuses before a signature even exists unless the seller is proven trustworthy.

Problem: Agents pay unknown x402 sellers blind.

HOW IT WORKS

1. Checks run in order and stop at the first failure: bad input, caller's price ceiling, seller's track record, then the live 402 is matched against what the caller expected (payee, network, asset, amount).
2. Only after all pass is the payment code even loaded.
3. Refusals use a closed list of reason codes.

WHAT MADE IT STAND OUT

- Proves refusals never reach the signer: the signer is wrapped and counts how often it is touched (0 on every refusal).
- Mutation testing: deliberately breaks the code six ways and requires the tests to catch each.
- Reports “settle claimed”, not “settled”, until the chain confirms it.
- Pre-registered A/B tests, including results that went against them.

What we take

Ordered checks with reason codes (same as our core) and **proof that refusals never touch the key**.

Part 3 · What the winners had in common

Fourteen design rules show up again and again. A project that follows most of them looks serious to any judge; we should be able to point at each one in our demo or README.

Rule	Seen in	What it means
Enforce where the money moves	Batas, Mandate, Harness, LedgerMind, Cordon	Limits checked by the program that moves funds. App-layer checks alone are “nowhere”.
The model never holds the key	Capsule, Cordon, TrueTick, GhostKey, LedgerMind	A separate signer; the AI can only ask. Prompt injection has nothing to grab.
The agent can't choose who gets paid or how much	Cordon, payOrRefuse, Turnstile, OpenBook	Price and payee come from the 402 or the chain and are matched against policy.
Refuse before signing, with reason codes	payOrRefuse, Cordon, GhostKey	A closed vocabulary of refusals the agent can act on.
Fail closed	Capsule, LedgerMind, Novi Corpus, payOrRefuse	Unknown, unreadable or missing means no.
Don't trust the facilitator; read the chain	Novi Corpus, payOrRefuse, TrueTick	Settled only after the transaction is read back.
Human approval bound to the exact purchase	AutoVoyage, Harness, GhostKey	Approve a hash of exactly what will be bought; elevated authority expires.
Kill switch that doesn't need our server	Capsule, Harness, Batas	Owner revokes directly on chain; everything stops at once.
Limit the blast radius in time	NotYet, Batas, GhostKey	Expiry and period budgets: a hack costs one period at most.
Public audit trail	AutoVoyage, Carpool, Batas, OpenBook	Every payment, refusal, grant and revoke is publicly readable.
Prove it with an experiment	Batas, Cordon, Carpool, payOrRefuse, LedgerMind	With vs without; attack runs; measured savings; honest negative results.
Label what is mocked	AutoVoyage, Mandate-app, TrueTick	Judges trust a project that says exactly what is real.
Free to look, pay only when needed	Batas, OnchainRouter, TrueTick	Discovery and pre-checks are free; never pay for what can't be delivered.
Legible to agents and judges	OnchainRouter, Batas, LedgerMind, Carpool	MCP, OpenAPI, llms.txt, one-command setup.

Part 4 · Our plan

Positioning

One-line promise (draft): “Your AI agents can spend on their own, and Solana guarantees they can never spend more than you allow, even if they are hacked.”

The use case still has to be chosen; the plan below works for any of our candidates. Choose it first, then rewrite the promise around that user.

The baseline: what we must ship (B1–B8)

Every winner had some version of these. Together they make a complete, believable product.

#	Feature	What it does	How on Solana	Learned from
B1	Budget tree with ancestor debit	Owner → team/project → agent; helps get less, never more; every spend counts all the way up.	Already in core/ (child ≤ parent, subtree revoke).	Cordon
B2	Cap enforced by Solana itself	Each agent spends from its own token account through an SPL delegate with a cap. Over the cap, the token program refuses.	SPL approve / revoke on devnet USDC; Squads spending limits if we need periods or recipient lists.	Batas, Harness
B3	Refuse-before-sign gate	Ordered checks with reason codes: budget, approved seller, max price, 402 matches expectation. Unknown = no.	Our server, before any signature; extends core/ codes.	payOrRefuse, Cordon
B4	The AI never holds a key and has no pay tool	Agent's only tool is fetch_paid(url, purpose). A separate signer pays, reading price and payee from the 402.	MCP server + signer process with an empty environment for the model.	Cordon, Capsule
B5	Real payments, verified from the chain	x402 on Solana devnet in USDC; marked settled only after reading the transaction back.	@x402/svm; RPC getTransaction.	Novi Corpus, payOrRefuse
B6	Receipt on every payment	Each payment carries a memo: budget id, purpose hash. Spending report built from the chain.	Memo instruction in the payment transaction.	LedgerMind, OpenBook
B7	One-tap approval for big purchases	Over a threshold, the owner gets a link; tapping it approves exactly that purchase.	Solana Blink (Action) carrying the purchase hash.	AutoVoyage, Harness
B8	Kill switch without our server	Owner revokes an agent from their own wallet in one transaction; others keep working.	SPL revoke signed by the owner directly.	Capsule, Harness

The standout features: what makes us unique (X1–X8)

Pick by effort. **X1, X3, X6 and X7 are small** and should all make it. X2 and X8 if time allows. **X4 and X5 are the roadmap slide.**

#	Feature	Why it's special	Effort	Inspired by
X1	"Turn off our server"	On stage we stop our backend and let a compromised agent try to pay directly. Solana still refuses because the delegate cap lives in the token program. The clearest possible answer to "why Solana".	Small	Batas
X2	Red-team arena	A prompt-injected agent tries to drain the budget live; a scoreboard counts attempts, refusals and dollars actually lost (target: \$0 beyond the cap). Same attacker with and without protection, Batas-style.	Medium	Batas, Cordon
X3	Blast-radius meter	Dashboard shows the most money an attacker could take right now (sum of live caps), and how revoke or a smaller period shrinks it. Turns safety into one number.	Small	NotYet
X4	Receipt per request (no double charge)	Each purchase creates a small on-chain receipt keyed by a hash of the request; paying twice for the same request is impossible and the seller releases data only once the receipt exists. Fixes the "free shopping" class from our research.	Large (Anchor program)	OpenBook, research \$3.1
X5	Money that carries its own rules	A Token-2022 wrapped USDC whose transfer hook checks approved sellers and per-payment limits on every transfer, wherever it goes.	Large	Our research \$5
X6	Chain-built invoice	One click turns a budget's memos into an invoice or expense report, every line linked to its transaction. Speaks directly to business judges.	Small	LedgerMind, OpenBook
X7	Don't pay twice + savings number	Before buying, check whether the team already bought it; show "€X saved this week".	Small	Carpool
X8	Spending advisor	An assistant reads the spending history and suggests revokes, smaller caps or blocked sellers, with the numbers it used.	Medium	LedgerMind

Why X1 is our headline

Every ETHOnline winner had to argue that its limits were safe. **We can show it live:** stop our server, let a compromised agent sign a payment directly, and watch the Solana token program refuse it because the cap is part of the token account itself. It answers "why Solana?" in ten seconds, and nothing at ETHOnline could do the same on Solana.

The demo (about 2 minutes)

1. **Setup (10 s):** owner gives a project €50 and two agents €20 each, approved sellers only, approval needed above €10.
2. **Normal work (30 s):** agent buys three items; each payment is a real devnet transaction with a receipt memo.
3. **Refusals (20 s):** unapproved seller → VENDOR_NOT_APPROVED; over budget → OVER_LIMIT. No money moved.
4. **Approval (15 s):** a €15 purchase waits; owner taps the Blink on a phone; it goes through.
5. **Attack (25 s):** prompt-injected agent tries to drain. Then we **turn off our server:** the agent signs directly, and Solana refuses (X1). Blast-radius meter shows the most it could ever take (X3).

6. **Kill switch (10 s):** owner revokes one agent from their wallet; the other keeps working.
7. **Report (10 s):** invoice built from the chain, each line linked to its transaction (X6).

How this scores against the WHU criteria

Criterion	What we show
A useful idea	One specific user and one plain promise; real incident numbers (NotYet, Cordon paper); the chosen use case.
A working prototype	B1–B8 live on devnet with explorer links; one-command setup; mocked parts labelled.
A clear role for Solana	B2 + X1: the token program enforces the cap even with our server off. B7 Blinks. B6 Memo. Sub-cent fees make per-call payments possible.
Potential to grow	Same safety layer fits every use case on our list; roadmap X4, X5; per-seat or per-payment pricing.

Timeline to the deadline

When	What
Fri 3 Oct, morning	Pick the use case and the one-line promise. Day-1 spike: x402 on Solana with an SPL delegate as payer (biggest technical risk). Fall back to a direct SPL transfer by the delegate if @x402/svm does not support it.
Fri 3 Oct, afternoon	B2, B5, B6 in chain/. B1, B3 codes in core/. B4 MCP + signer in surface/.
Fri 3 Oct, evening	End-to-end run on devnet: three purchases, one refusal, one revoke. Record a backup video.
Sat 4 Oct, morning	B7 Blink approval, B8 owner revoke, X1 server-off demo, X3 blast-radius number, X6 report.
Sat 4 Oct, afternoon	X2 red-team run (if time). Deck: problem with numbers → promise → demo → why Solana → growth. README with what is real and what is mocked.
Sat 4 Oct, evening	Final demo video, submit before midnight CEST. Do not start X4 or X5; they go on the roadmap slide.

Risks and fallbacks

Risk	Plan
@x402/svm may not accept a delegate as the payer	Test first thing Friday. Fallback: our signer sends the SPL transfer itself with the delegate authority, and the seller verifies by reading the transaction.
Devnet USDC / faucet limits	Get devnet USDC on Friday morning; keep a second funded wallet.
Scope creep	Only B1–B8 are required. X ideas only after an end-to-end run works.
“This already exists” (Cordon, LedgerMind)	Answer: on Solana the cap lives in the token program, approvals are Blinks, and the report is built from the chain; plus the use case.
Overclaiming	Label anything mocked; put measured numbers only.

For later competitions

- **Keep the baseline as the product.** It is chain-specific in the right way (Solana enforces) and use-case agnostic.
- **Swap the story per competition:** the same core with a different first user (fleet, research, procurement, family).
- **Grow into X4 and X5** (receipt per request, money with its own rules) for infrastructure-focused tracks; these are where our research found open problems.
- **Keep a RESULTS.md** mapping every claim to evidence, as our Hedera version did; winners like Batas and payOrRefuse made this their strength.

No plan guarantees a win, but this one covers what every judge at ETHOnline rewarded, and adds what only Solana can show.